

# FFmpeg Architecture & Internals

*Understanding the Media Swiss Army Knife*

With Fracture Project Integration

VideoClassifier • July 2026

# is FFmpeg?

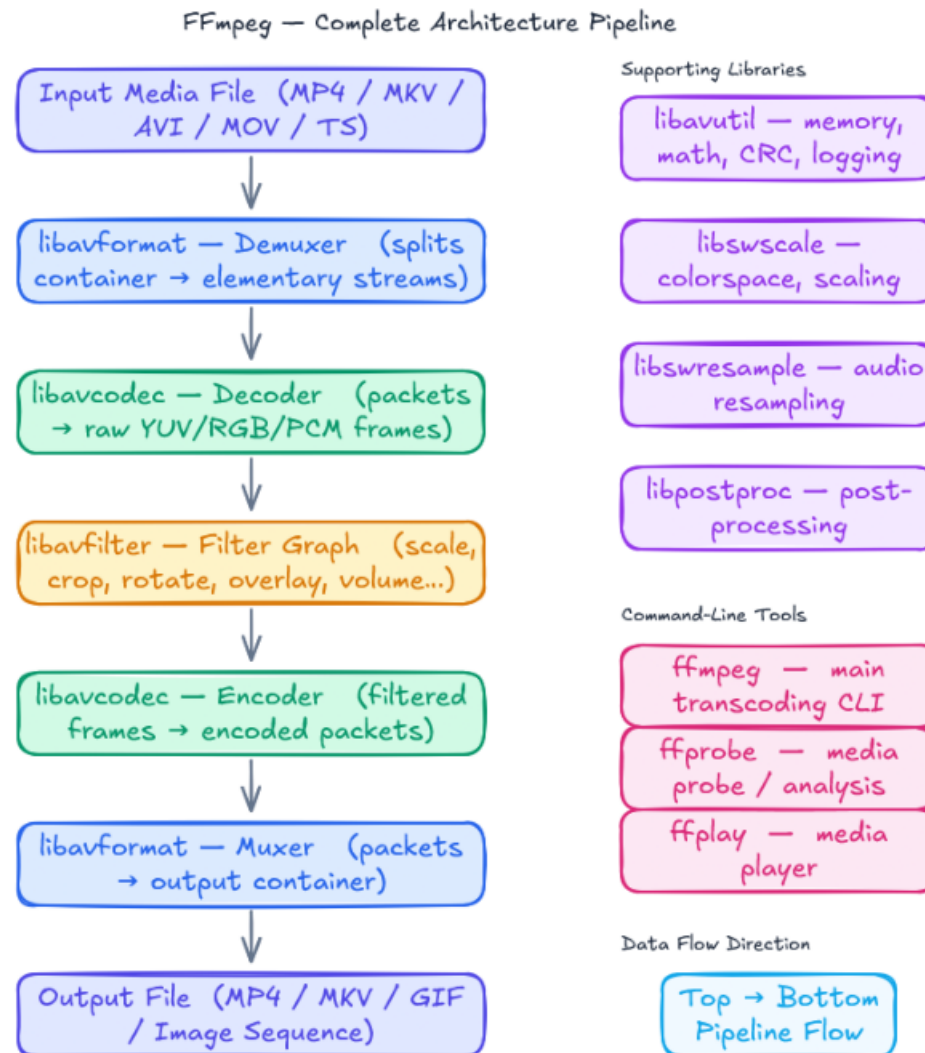
*s-platform multimedia framework*

- ▶ FFmpeg stands for "Fast Forward MPEG"
- ▶ Founded by Fabrice Bellard in 2000; now the industry-standard media framework
- ▶ Supports virtually all codecs (H.264/AVC, H.265/HEVC, AV1, VP9, AAC, MP3, FLAC...)
- ▶ Supports all major containers (MP4, MKV, AVI, MOV, TS, WebM, FLV...)
- ▶ Cross-platform: Windows, macOS, Linux, BSD, Android, iOS
- ▶ Licensed under LGPL/GPL; the most widely used multimedia library in the world
- ▶ Used by every major media player, browser, streaming platform, and video editor

- ▶ FFmpeg is the backbone of Fracture — ffprobe extracts metadata, ffmpeg handles all transcoding

# eline Architecture

a linear chain of processing stages



# Files: I/O & Codec Layers

*Format + libavcodec*

## libavformat -- Demuxer & Muxer

- ▶ Handles container I/O: reads/writes MP4, MKV, AVI, MOV, TS, FLV, etc.
- ▶ AVFormatContext is the top-level I/O context; manages streams, packets, metadata
- ▶ Demuxer: splits container into elementary streams (video, audio, subtitles)
- ▶ Muxer: interleaves encoded packets into output container
- ▶ Key structs: AVFormatContext, AVStream, AVPacket, AVIOContext

## libavcodec -- Encoder & Decoder

- ▶ The largest and most complex FFmpeg library: ~500k+ lines of C
- ▶ Supports H.264, H.265/HEVC, AV1, VP9, MPEG-4, AAC, MP3, FLAC, Opus...
- ▶ AVCodecContext controls codec parameters; AVCodec defines the codec implementation
- ▶ Hardware acceleration: NVENC (NVIDIA), AMF (AMD), QSV (Intel), VA-API, Vulkan
- ▶ Encoding flow: raw frame → packetized bitstream → access units
- ▶ Decoding flow: AVPacket (compressed) → AVFrame (raw YUV/RGB/PCM)

# Filters: Filters & Processing

*libswscale, libswresample, libavutil*

## **libavfilter -- Filter Graph**

- ▶ Graph-based filter processing: buffersink and buffersrc form the filter chain
- ▶ Scale, crop, rotate, overlay, fade, drawtext, volume, equalizer, trim, concat...
- ▶ Complex graphs: video and audio filters can be chained in arbitrary topologies
- ▶ AVFilterGraph manages the complete filter chain; filters linked via AVFilterLink

## **libswscale, libswresample, libavutil**

- ▶ libswscale: pixel format conversion (YUV↔RGB), image scaling and resizing
- ▶ libswresample: audio format conversion, sample rate conversion, channel remapping
- ▶ libavutil: memory management (AVBuffer), math, CRC, hashing, logging, timestamps (AVRational)
- ▶ libpostproc: older post-processing filters (deblocking, deringing); largely superseded by libavfilter

# ough FFmpeg Pipeline

from input file to output file

|    |                     |                                                                                           |
|----|---------------------|-------------------------------------------------------------------------------------------|
| 1. | <b>Open Input</b>   | avformat_open_input() reads the file header and populates AVFormatContext                 |
| 2. | <b>Find Streams</b> | avformat_find_stream_info() probes all streams for codec parameters                       |
| 3. | <b>Open Decoder</b> | avcodec_open2() initializes the decoder with AVCodecContext                               |
| 4. | <b>Read Packet</b>  | av_read_frame() reads one AVPacket (compressed data) from the demuxer                     |
| 5. | <b>Decode</b>       | avcodec_send_packet() + avcodec_receive_frame() decodes packet into raw AVFrame           |
| 6. | <b>Filter</b>       | av_buffersrc_add_frame() pushes into filter graph; av_buffersink_get_frame() pulls result |
| 7. | <b>Encode</b>       | avcodec_send_frame() + avcodec_receive_packet() encodes filtered frame to AVPacket        |
| 8. | <b>Write</b>        | av_write_frame() writes the encoded packet to the output container via the muxer          |
| 9. | <b>Cleanup</b>      | av_write_trailer() finalizes output; all contexts freed with avformat_close_input()       |

# t: FFmpeg Integration

Images FFmpeg for video understanding

## ffprobe Usage

- ▶ Fracture calls ffprobe via subprocess for every input video
- ▶ Extracts: duration, codec, resolution, bitrate, FPS, pixel format
- ▶ Parses JSON output into a structured metadata dictionary
- ▶ Fields: width, height, codec\_name, r\_frame\_rate, duration, bit\_rate
- ▶ Probes both video and audio streams independently
- ▶ Metadata used for filtering, logging, and downstream CLIP analysis

## CLIP Embedding Pipeline

- ▶ Extracts frames from video using OpenCV VideoCapture
- ▶ Resizes to 224x224 and normalizes using ImageNet statistics
- ▶ Feeds frames through CLIP ViT-B/32 vision encoder
- ▶ Generates 512-dimensional embedding vectors per frame
- ▶ Full frame extraction runs ffmpeg subprocess (quality, fps control)
- ▶ Embeddings stored as .npy files per video for similarity search

# Command-Line Tools

*the three pillars of the FFmpeg ecosystem*

## ffmpeg

The main transcoding CLI. Convert, filter, trim, concat, stream.

```
$ ffmpeg -i input.mp4 -c:v libx264 -crf 23 -c:a aac output.mp4
```

```
$ ffmpeg -i input.mp4 -vf "scale=1280:720,drawtext=text='Hello':fontsize=24" output.mp4
```

```
$ ffmpeg -i input.mp4 -ss 00:01:00 -t 30 -c copy clip.mp4
```

## ffprobe

Multimedia stream analyzer. Probes containers, streams, packets.

```
$ ffprobe -v quiet -print_format json -show_format -show_streams input.mp4
```

```
$ ffprobe -v error -show_entries stream=codec_name,width,height,r_frame_rate input.mp4
```

```
$ ffprobe -v error -count_frames -show_entries stream=nb_read_frames input.mp4
```

## ffplay

Portable media player built on FFmpeg + SDL.

```
$ ffplay input.mp4
```

```
$ ffplay -vf "eq=brightness=0.1:saturation=1.5" input.mkv
```

```
$ ffplay -f lavfi -i "rgbtestsrc=size=1920x1080:rate=30"
```



# Key Takeaways

FFmpeg is the industry-standard multimedia framework powering Fracture

- ✦ Pipeline: Input → Demux → Decode → Filter → Encode → Mux → Output
- ✦ Core libraries: libavformat, libavcodec, libavfilter, libswscale, libswresample
  - ✦ Fracture uses ffprobe for video metadata extraction
- ✦ Fracture uses ffmpeg and OpenCV for frame extraction and CLIP embedding pipeline
  - ✦ Cross-platform, open-source, and infinitely extensible

Built by the Fracture Team • July 2026

**Thank you**